

SURF SCHOOL MANAGEMENT SOFTWARE

Smart Surf School Management System

Everything a surf school runs on, in one application:
lessons, bookings, equipment, surf conditions, safety, finance and AI.

Version 1.0.0 · Django 5.2 LTS · Turkish / English

28

modules, one database

2,128

tests passing, 0 failing

39/39

screens working

THE PROBLEM

How many places does a surf school run in?

A typical day: bookings arrive on WhatsApp, student details live in a notebook, nobody is sure who has which board, takings are on paper receipts, and the forecast is in another app entirely.

- **Two lessons, one slot** Nobody sees the clash until two coaches wait for the same student.
- **Missing equipment** Who has the board? When is it due? What about the deposit?
- **Safety details too late** The instructor learns about the asthma after the lesson, not before.
- **Invisible money** What the month actually earned is a guess.
- **Knowledge that walks out** When someone leaves, what they knew leaves too.

This system puts all five in one database.

MEASURED

The system in numbers

Every figure below was read from the project — none is an estimate.

28

modules

86

data models

75

REST resources

15

roles

203

capabilities

2,128

tests passing

39/39

screens working

20

acceptance steps

Tests via `pytest`: 2,130 collected, 2,128 passing, 2 skipped because they need a live AI provider. Code coverage was NOT measured for this release, so no percentage is quoted. Screens verified with real HTTP requests against a running server.

SCOPE

Six areas, one application

People

Customers, students, instructors, CRM. Surf level, medical notes, certification expiry, commission.

Operations

10 lesson types, booking calendar, waitlist, surf camps, the day's schedule.

Equipment

QR-labelled inventory, hourly/daily/weekly rentals, damage and service history.

Surf & safety

Live wave, wind and tide, a Surf Score per level, incident reports, lifeguard roster.

Money

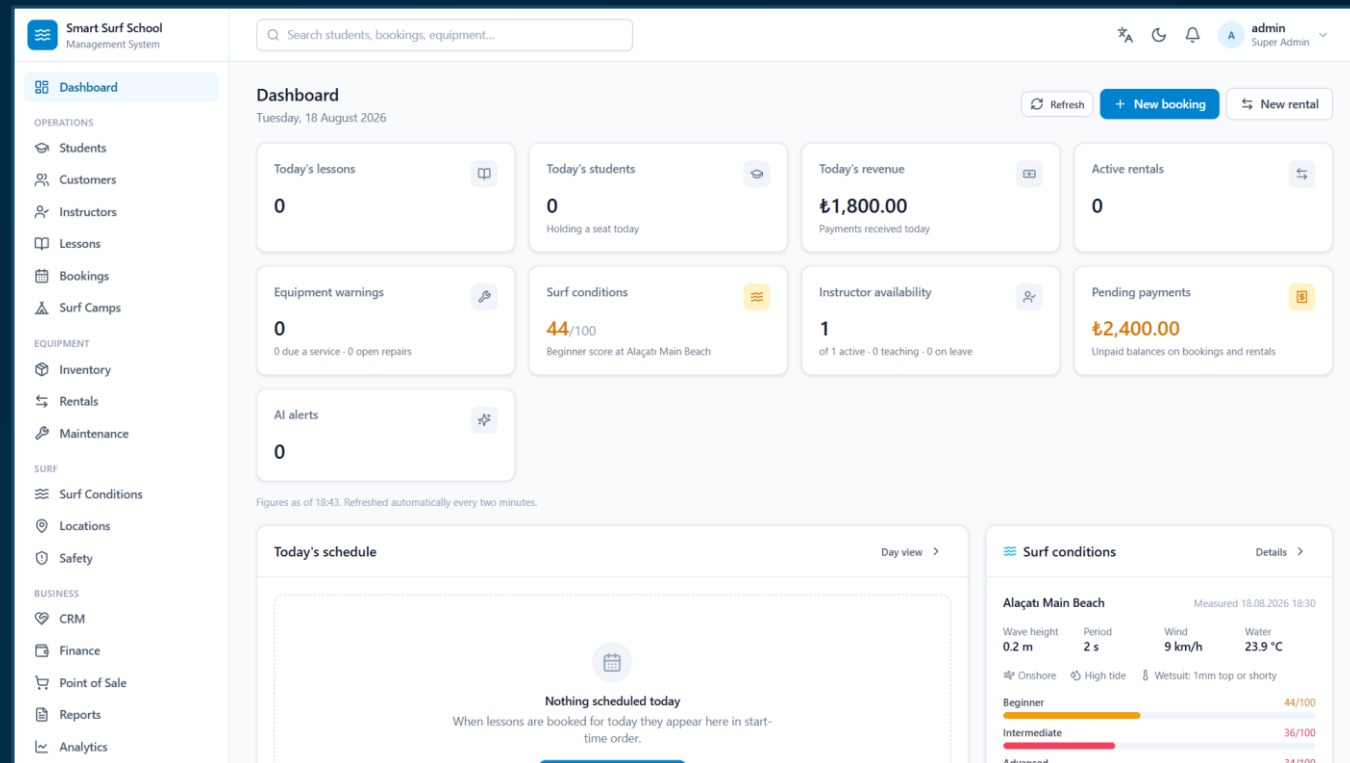
Payments, refunds, invoices, expenses, commission, lesson packages and point of sale.

Decision support

Statistics engine, dashboards, PDF/Excel/CSV reports, backups and an audit trail.

THE INTERFACE

What it looks like: the dashboard



Today's lessons, revenue, pending balances, equipment warnings and sea conditions — at a glance.

Every screenshot in this deck was taken from the running application on 2026-08-18, showing synthetic demo data — none of them is a mock-up or a design rendering.

Who can see what?

15 roles and 203 capabilities come from one table. The menu, the screens, the REST API and the AI assistant all read it — which is why a screen can never offer an action the API would refuse.

Super Admin	Everything, including backup restore
Manager	Operations + finance + staff
Head Instructor	Lessons, instructors, safety sign-off
Surf Instructor	Their own lessons and students
Lifeguard	Safety module, incident reporting
Reception	Customers, bookings, taking payment
Rental Staff	Check-out and check-in, deposits
Finance	Payments, refunds, reports, commission
Customer / Student	Only their own records — enforced per row

Taking money (finance.view) and reading the takings (finance.revenue) are separate privileges; rental staff hold the first and not the second, so the aggregate query is never run. Asserted in apps/accounts/tests/test_object_scoping.py.

BOOKINGS

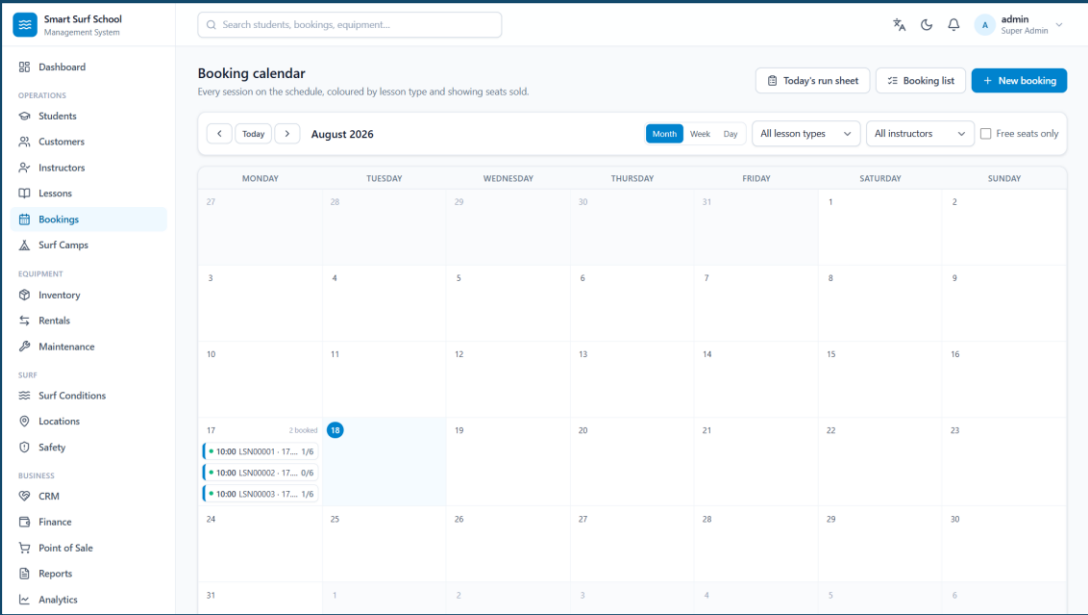
The system catches the clash, not the person

Rules run live while a booking is typed. Each one is tested separately.

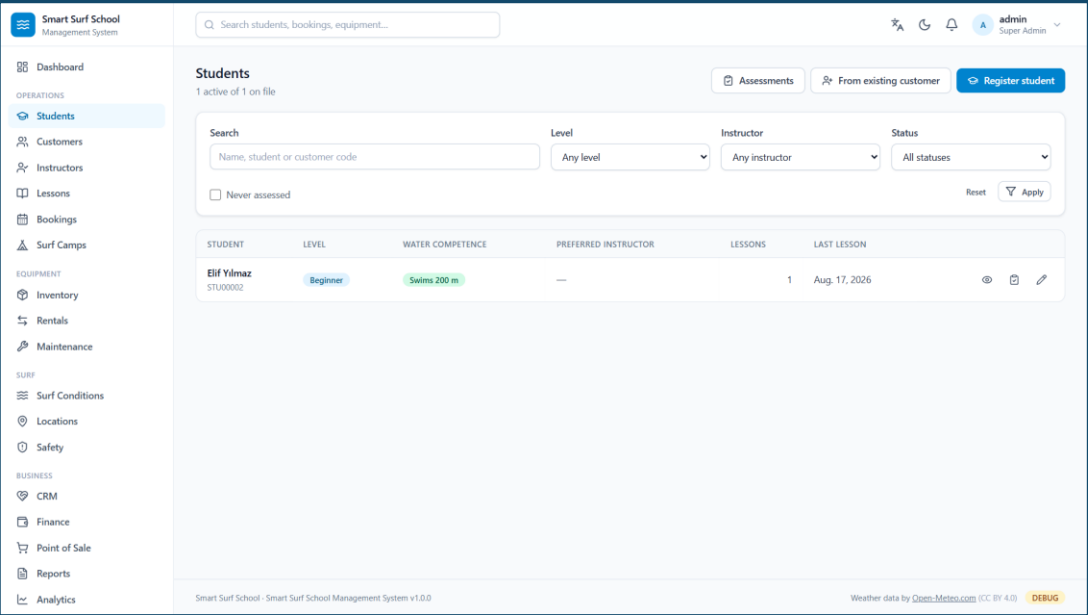
- **Free seats** Is the lesson full, how many places are left.
- **The student's own clash** Are they already booked at that time.
- **Level fit** Does the lesson type accept this level.
- **Instructor availability** Weekly hours, time off, another lesson.
- **Instructor-to-student ratio** A stricter limit applies when a minor is in the group.
- **Safety restrictions** Any medical or skill restriction on the student.

When something clashes, the system says exactly what is wrong — and the rule cannot be worked around, because that is when people get hurt.

The calendar and the student file



Booking calendar: coloured by lesson type, seats sold on every entry.



Student list: level, water competence and last lesson side by side.

Who has the board, and when it is due

QR labels

Every item has a unique code and a QR label.
Scan it at the counter and the line is added.

The right board

Volume in litres from weight and level;
wetsuit thickness from the water
temperature.

Honest pricing

Hourly, daily or weekly — and a hire of a
week or more automatically takes the weekly
rate when it is cheaper.

Late and damaged

Late fees are capped at three times the hire;
reported damage opens a maintenance
record automatically.

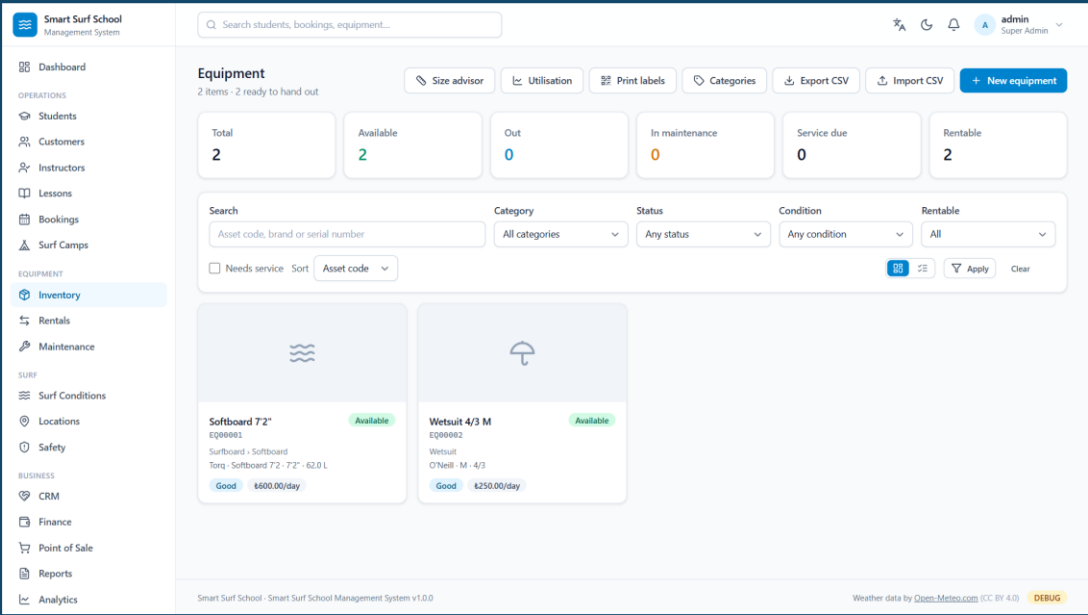
Service history

Ding, crack, fin, leash, wetsuit tear — every
repair recorded with its cost.

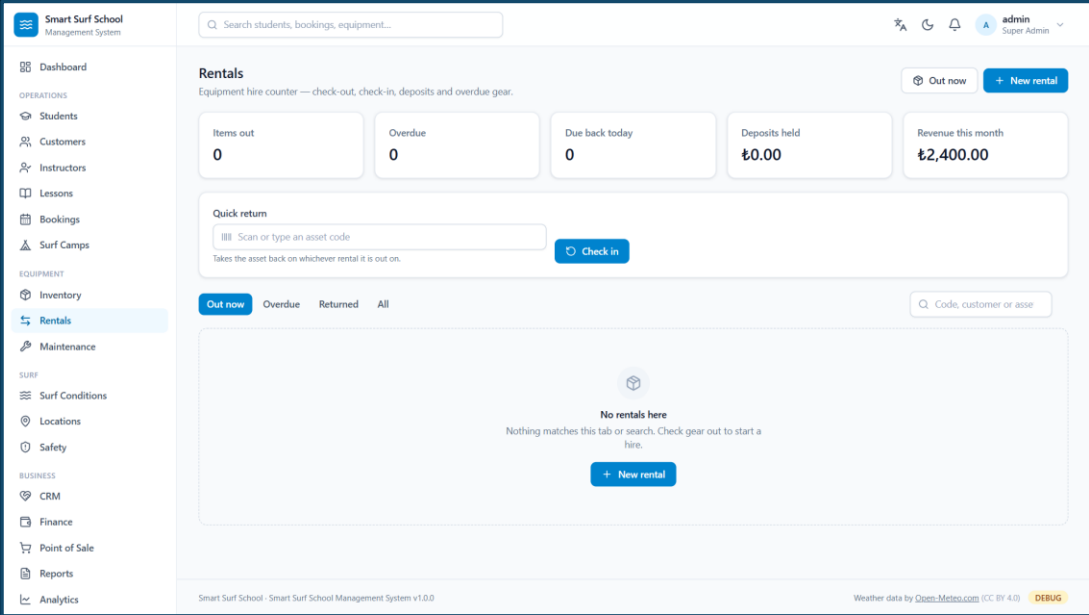
Predicted maintenance

A risk ranking computed from your own
service history. Statistics — not an AI guess.

Inventory and the hire counter



QR-tagged inventory: status, condition and daily rate on the card.



The hire counter: one scan to check gear back in, deposits and overdue tracking.

SURF CONDITIONS

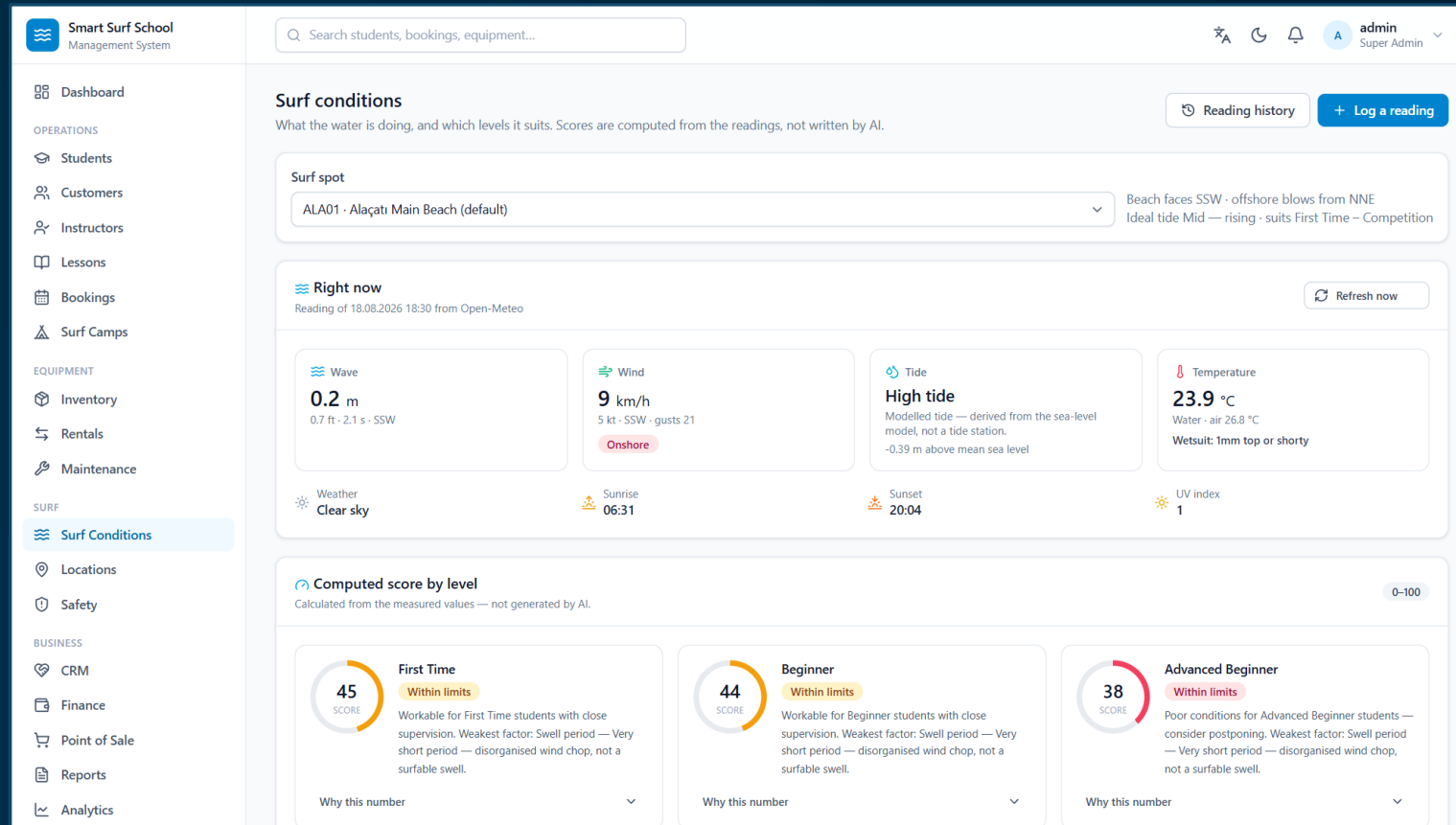
Surf Score: 0–100, and why it is that number

Wave, wind, period, tide, water temperature and weather, scored separately for each surf level. The score is COMPUTED — never asked of a language model — and every component is shown.

Wave height fit	35%
Wind quality (offshore/onshore)	25%
Swell period	15%
Tide match	10%
Weather and precipitation	10%
Water temperature comfort	5%

Hard safety gate: above the level's wave or wind limit the score is capped at 25 and marked unsafe. With no wave data no score is produced at all — an unknown ocean is not a safe ocean.

Live sea state, computed score



A live Open-Meteo reading: wave, wind, tide and water temperature — and the Surf Score computed per level, with its reasoning shown.

The AI is never the final authority on a safety decision.

The system summarises conditions, thresholds and history. Whether a lesson runs, whether a student enters the water, and whether a piece of equipment is fit for use is decided by a qualified member of staff.

The data model works the same way: a weather warning suggested by the AI is not an active warning until a named person acknowledges it.

Exact to the cent, with the history intact

Decimal arithmetic

Money is never a floating-point number anywhere. 12 digits, 2 decimals — sums and comparisons are exact.

Refunds keep the record

A refund creates a matching negative entry; the original payment is never altered.

Atomic operations

A booking and its lesson attendance are created together or not at all.

Lesson packages

Sell a discounted multi-lesson package; it draws down as it is used and the balance shows on the customer's page.

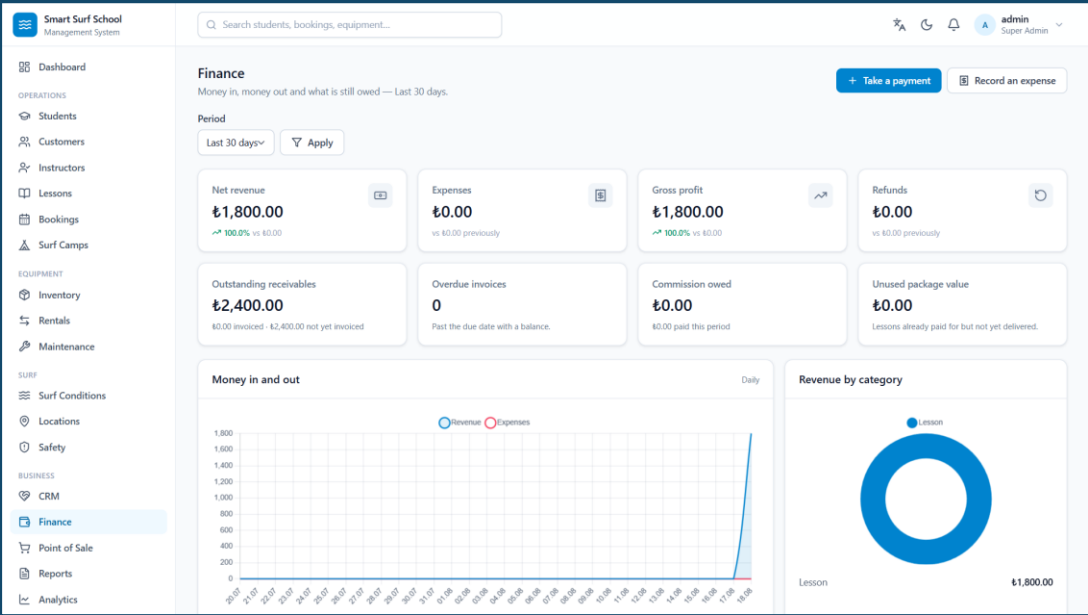
Point of sale

Barcode entry, a stock ledger, printed receipts. Voiding a sale returns the stock rather than deleting the record.

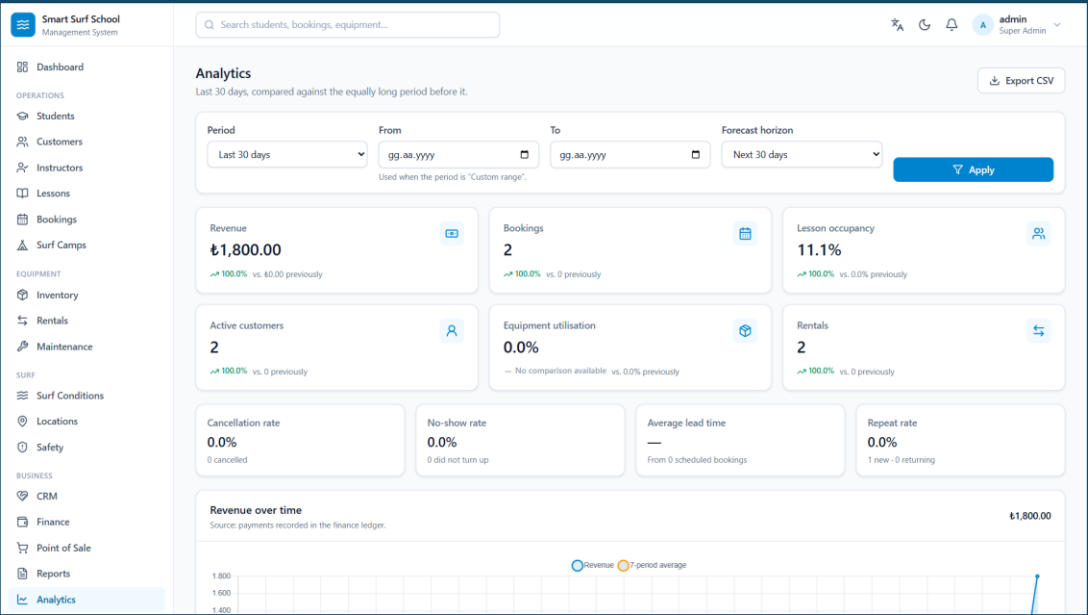
Audit trail

Every payment, refund and permission change is stored immutably with who, what and when.

The finance board and analytics



Net revenue, expenses, receivables and overdue invoices — last 30 days.



Occupancy, cancellation rate, repeat customers and the revenue trend.

Measured AI, not claimed AI

Model choices came from real calls made on this machine, not from vendor documentation.

857 ms

Nemotron 3 Super
(assistant)

535 ms

Nemotron 3.5 Lightning
(fast routing)

389 ms

Riva Translate
(TR ↔ EN)

311 ms

Embedding 1B
(2048 dimensions)

A finding that changed the design: without `thinking: false` the Nemotron models exceeded 90 seconds and leaked chain-of-thought into the answer. With the flag: 535 ms and a clean reply. See [docs/research/VERIFIED_API_PROBES.md](https://docs.research/VERIFIED_API_PROBES.md)

Local first, cloud optional

Providers sit behind one interface. Call sites ask for a ROLE, never a model name. A failed provider is a value rather than an exception — so the application stays fully usable with no internet at all.

LM Studio (local)	Free, offline, data never leaves the machine
NVIDIA NIM	Strong reasoning, vision, translation, embeddings
Anthropic Claude	Optional
OpenAI-compatible	Ollama, vLLM and others
Local only	For when customer data must not leave the building
Automatic	Cheap questions stay local, hard ones go to the cloud

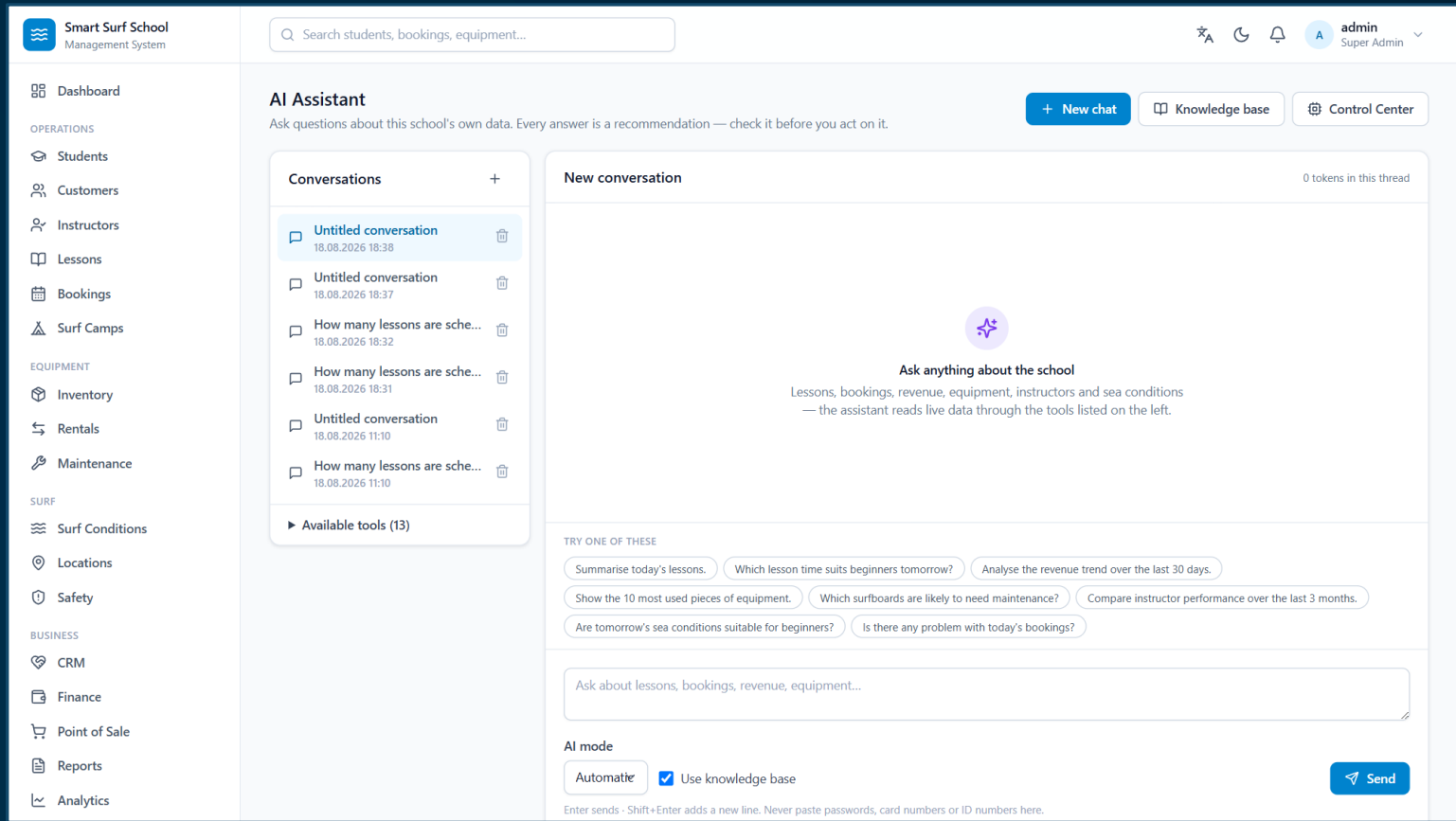
A local embedding model means knowledge-base search works offline too.

The assistant cannot invent data

«Do not make things up» is a mechanism here, not a hope. The model can only obtain a figure from a tool that runs a real query.

- **13 database tools** Lessons, bookings, revenue, equipment, maintenance, rentals, instructors, customers, surf conditions, safety.
- **Permissions checked in the tool** A tool runs with the capabilities of the person ASKING. The assistant is not a privilege-escalation path.
- **«No data» is a real answer** When a query returns nothing the model says so rather than filling the gap with a plausible number.
- **Citations** Answers drawn from the knowledge base show the document they came from.
- **Visible working** Under every answer: which provider, which model, how many milliseconds, and which lookups ran.

The assistant at work



The AI assistant talks to the school's own data: 13 database tools, ready-made questions, and a provider/model/latency line under every answer.

The AI proposes, policy decides, a human approves

No shell

Commands run as a validated argument vector. Chaining and redirection are not filtered — they are impossible.

Allowlist

git status yes, git push no. manage.py test yes, manage.py flush never.

Workspace jail

UNC paths, drive-relative paths, 8.3 short names, CON/NUL device names and NTFS streams are all closed off.

Approval gate

An edited command is re-validated from scratch — approval is not permission to bypass the policy.

Clean environment

No credential reaches the child process; on timeout the whole process tree is killed.

Full record

Every decision is audited, including the refusals.

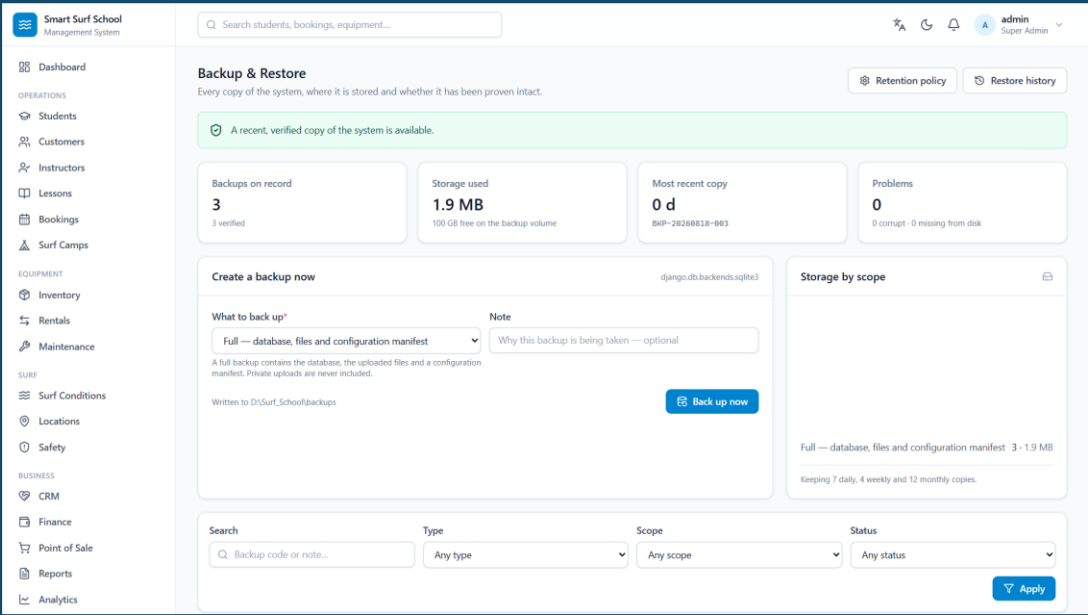
Backups: a plan, not a hope

Taking a backup is the easy part; the real risk is restoring the wrong one over good data. Restore is deliberately made difficult.

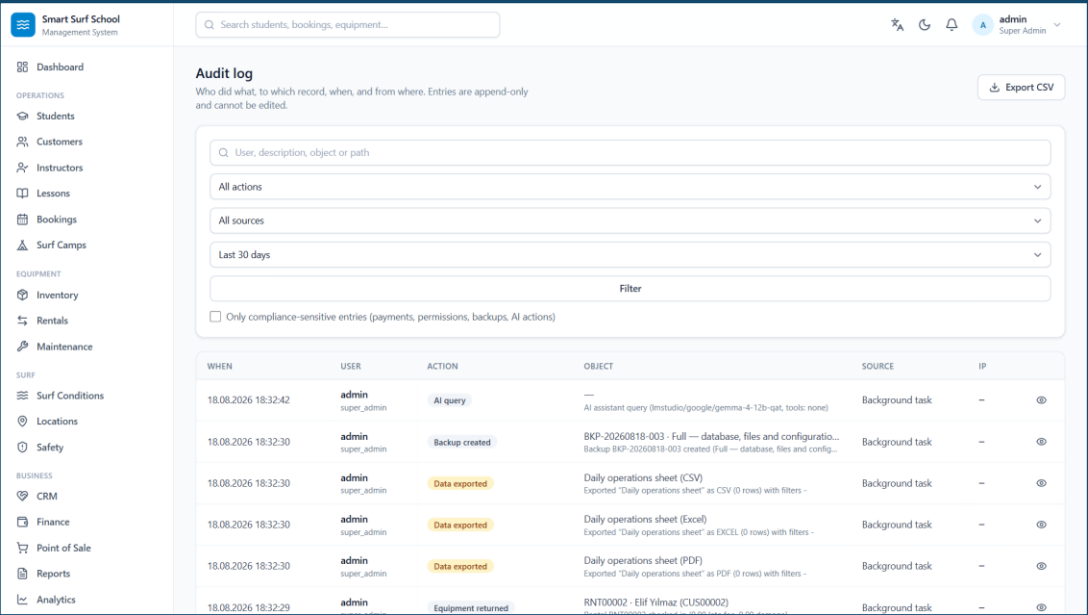
- | | |
|------------------|--|
| 1. Verify | SHA-256 checksum and an integrity check |
| 2. Type the code | The operator types the backup code to continue |
| 3. Permission | backups.restore is required separately |
| 4. Safety backup | The current state is backed up FIRST |
| 5. Restore | With every step written to the audit log |
| 6. On failure | Everything is put back automatically |

SQLite is captured through the sqlite3 backup API rather than by copying the file — copying a live database produces a corrupt backup.

Backups and the audit log



Verified backups, the retention policy and a one-click new copy.



The audit log: who, what, when — rows are appended, never edited.

Easy to learn, in two languages

Turkish and English

Menus, notifications, validation messages and Help Center and Training Center content is genuinely bilingual. The interface chrome is English today: 337 templates use {% trans %} but no compiled Turkish catalogue ships yet. Turkish locale formatting -- dates, numbers, currency -- does apply.

Setup wizard

First run asks for the school name, currency, language and surf spot — and then creates the real records.

Help Center

16 categories, 22 articles — a reference for every screen, in both languages.

Role aware

An instructor's dashboard leads with their own lessons; a customer sees only their own bookings.

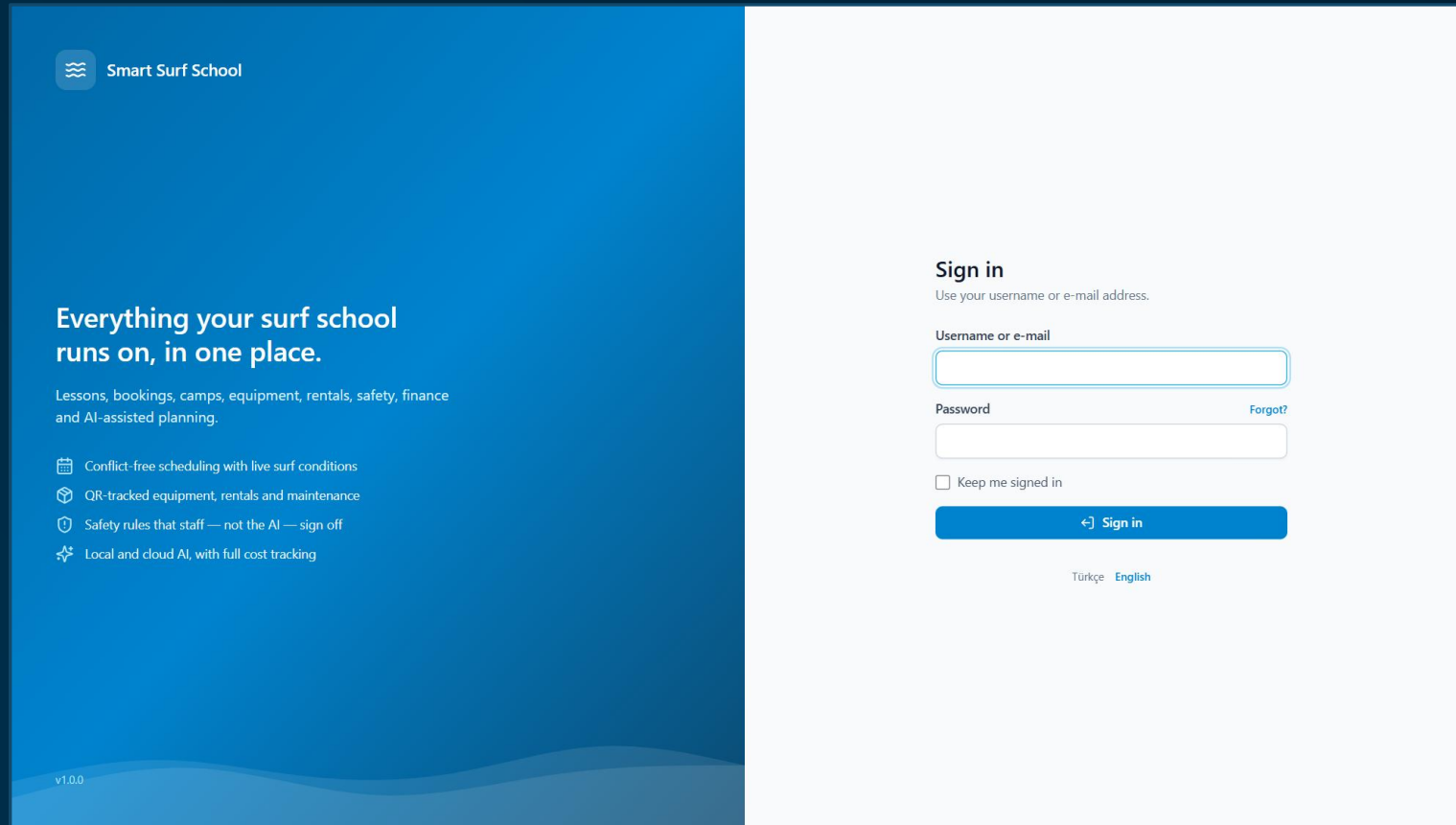
Training Center

10 interactive courses, 71 steps. Real tasks, from your first student to your first backup.

Dark and light

Bright at the counter, dark in the office.

Bilingual sign-in, role-aware screens



The sign-in screen: TR/EN language switch; after sign-in the menu and dashboards shape themselves to the user's role.

TECHNOLOGY

Plain, durable, portable

The technology was chosen for this business's real conditions — one computer, weak internet, no IT staff — rather than for fashion.

Python 3.11 + Django 5.2 LTS	Supported until 2028
Django REST Framework	75 resources, OpenAPI documentation
HTMX + Alpine + Tailwind	No separate build, no CDN
SQLite → PostgreSQL	One environment variable
Redis / Celery	Used when present; the app is identical without
Works offline	Every asset is vendored locally

The dependency audit reports no known vulnerabilities, and static security analysis reports no high-severity findings.

QUALITY

Whatever is claimed was measured

All of the following were run on this machine and the results recorded.

2,128

tests passing
0 failing

90

access-control
assertions

39/39

screens
rendering

20

end-to-end
acceptance steps

The acceptance scenario runs through the real service layer: create an instructor and a student, a lesson, a booking, take payment, complete the lesson, hire equipment and take it back, generate reports, make a backup, ask local and cloud AI, and check the audit log.

HONESTY

Known limits

If an introduction only lists strengths, the buyer discovers the weaknesses in production. Here are the known ones:

- **Development runs on SQLite** PostgreSQL is the production target and the switch is one environment variable, but PostgreSQL was not installed on this machine.
- **No browser automation suite** Screens are verified over real HTTP; click-through flows were tested by hand.
- **Open-Meteo's free tier is non-commercial** The data licence is fine, the free hosted service is not; a met.no fallback is built in.
- **No load testing** The target is one school on one machine.
- **AI providers are tested by hand** Automated tests never touch the network — that is a deliberate decision.

IN SHORT

A surf school's day, in one application

- The system catches the clash and the safety rule
- Money is exact and the history cannot be rewritten
- The surf score is computed, never asked of an AI
- The assistant only ever talks about real data
- It works with no internet at all

Version 1.0.0 · github.com/Azizsekerdil/smart-surf-school-management-system

Windows 10/11 (x64) & macOS (Apple Silicon) · Download: [GitHub Releases](#)

(v1.0.0)

The macOS build targets Apple Silicon (arm64) and is not notarized — first launch: right-click → Open.